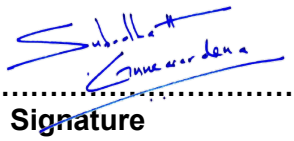
1. **Title of the journal paper (*Note: The title of the journal paper cannot be same as the title of the final-year undergraduate project. However, the title can be an important work related to the final-year undergraduate project*):**

A High-Fidelity Open-Source Digital Twin Framework for Multi-UAV Swarm Simulations

2. **Names and register numbers of the students:**

G.P.W.P. Wijerathne	EG/2021/4871
J.H.A.D.C. Epaladeniya	EG/2021/4504
K.A.G.S. Randil	EG/2021/4745
W.P.A.N. Fernando	EG/2021/4512

I hereby confirm that the above students have discussed with me and have finalized the above title and the main contents of the journal paper.

Dr. Subodha Gunawardena		07-08-2026
Supervisor's name	**Signature**	**Date**
Co-supervisor's name (If any)	**Signature**	**Date**

(***Note: The filled sheet should be submitted with the journal paper***)

A High-Fidelity Open-Source Digital Twin Framework for Multi-UAV Swarm Simulations

G.P.W.P. Wijerathne, J.H.A.D.C. Epaladeniya, K.A.G.S. Randil and W.P.A.N. Fernando

***Abstract*—Unmanned Aerial Vehicles (UAVs) are being increasingly used in coordinated multi-agent systems for performing complex tasks such as surveillance, disaster response, environmental monitoring, and precision agriculture. Multi-UAV operations offer advantages over single-UAV systems in terms of coverage, robustness, time efficiency, simultaneous actions and scalability; however, testing and analysing such systems in real-world environments is quite challenging due to safety risks, high costs, regulatory constraints, and the complexity of wireless communication networks. As a result, simulation based evaluation has become an essential step in the development of multi-UAV systems. This project presents the development of a multi-UAV simulation framework using open-source tools, integrating ArduPilot Software In The Loop (SITL) for UAV flight control, Gazebo for realistic 3D environment simulation, Robot Operating System (ROS) 2 as middleware for system integration, and Network Simulator-2 (NS-3) for realistic communication network simulation. The framework provides multi-UAV operation, Micro Air Vehicle Link (MAVLink) based communication, and integration of higher level applications such as dynamic clustering. The simulation platform is used to evaluate performance metrics such as communication latency, packet loss, bandwidth usage, system scalability and feasibility of executing a mission under varying network conditions. The resulting framework provides a highly scalable, fully reproducible testbed to advance research in distributed multi-UAV networks.**

***Index Terms*—Autonomous aerial vehicles, Swarm Robotics, Digital Twins, Ad hoc networks**

I. Introduction

A. Background and Motivation

Unmanned aerial vehicles (UAVs) expands its applications in a diversified area such as surveying and mapping [1], search and rescue [2] and many other applications [3]. With the recent advancements of UAV technology instead of using single-UAVs, usage of multi-UAVs has been evolving and identifies as a highly interested field in research. This high interest in multi-UAV systems have several advantages comparing to single-UAV systems. Time efficiency: the operations times of the missions can be significantly reduced as each UAV will complete a specific section of the mission, hence the work done by each UAV is reduced. This is immensely in time critical applications such as target search and urgent detection of nuclear radiation in a disaster [3]. Cost: having a single-UAV could be an expensive solution, instead having multiple UAVs could make the application cost more cheap by reducing costs related to power consumption. Having a team of smaller UAVs will be a cheaper alternative than using a single UAV having a heavy weight for applications such as a airdrop system. Simultaneous actions: in a team of UAVs can fulfill tasks in different geographical locations at the exact same time where a single UAV is definitely being incapable. Multi-UAV systems is a growing solution for this significant problem of continuous coverage. Fault tolerance: if only a single UAV is considered the loss of the UAV is simply the loss of the mission. When multi-UAV systems are used the loss of a single or several UAVs could be compensated by the remaining UAVs in the the multi-UAV system such as a search and rescue mission which is extremely impossible to cancel in middle.

On top of these advantages it is identified that there are a several disadvantages which limits the research regarding to multi-UAV systems. Legal restrictions are imposed in some jurisdictions which does not permit to fly in swarms [4]. Also the safety concerns regarding safe piloting among swarms has caused a severe problem. Such limitations, causes to drop research in the physical domain which paves path to make resarch more strong on simulation frameworks regarding multi-UAV systems.

B. Problem Statement

UAVs are incrasingly deployed in mission critical applications while the development of single-UAV systems was maturing consequently, recent research emphasises multi-UAV sytems that offer advantages like redundancy and cost effectiveness for cooperative goals. However, the real world deployment and testing of these networked multi-UAV systems are severely constrained by strict flying regulations, implementation cost, safety concerns and limited onboard analytics. Simulation becomes an ideal solution to mitigate the overheads of real world testing, many such existing simulation tools fall at instances because they often focus solely on either the accurate modelling of UAV operations [5] or communication and network dynamics [6].

This project addresses the challenge of accurately modelling complex, coupled systems which includes physical, network and easily controllable systems. Specifically, there is a lack of readily available, accessible, modular and extensible open-source frameworks capable realistically

G.P.W.P. Wijerathne, J.H.A.D.C. Epaladeniya, K.A.G.S. Randil, and W.P.A.N. Fernando are with the Department of Electrical and Information Engineering, Faculty of Engineering, University of Ruhuna, Sri Lanka

Corresponding author: G.P.W.P. Wijerathne (e-mail: wimukthiwijerathne@gmail.com).

simulating the dynamics of multi-UAV swarms performing distributed tasks, especially those involving onboard data processing and clustering. Therefore, there exists a clear need for a comprehensive digital twin system that integrates leading open-source tools to accurately evaluate key performance metrics, such as speed, latency and power requirements related to real-time data processing performed within a dynamic multi-UAV network.

C. Research Objective

Development of a high-fidelity framework for multi-UAV swarm simulation which is fully expandable and could be used for areas of multi-UAV research. The framework integrates ArduPilot, an open source UAV flight controller, Gazebo, a 3D environment simulator and using ROS2 as the middleware, which is capable of simulating realistic multi-UAV swarm systems.

II. Related Work

A. Open-Source Simulation Tools

The final outcome is the integration of four open-source tools which makes the framework more robust and easily handled.

ArduPilot(SITL): this enables the creation and use of unmanned vehicle systems which provides a large set of tools which is suitable for almost any vehicle and application. Since this is an open-source tool, this is being constantly evolving. ArduPilot copter is the full featured multicopter UAV controller dedicated UAV systems. Copter is capable of the full range of flight requirements from fast to smooth complex missions. Arducopter facilitates several frame types and Arducopter-Quad which consists of four copters [7].

Gazebo: this is most oftenly used for robot simulation environments. It offers the ability to accurately and efficiently simulate populations of robots in complex indoor and outdoor environments. This tools provides a robust physics engine, high-quality graphics and convenient programmatic and graphical interfaces. Includes several features such as dynamic simulation, advanced 3D graphics, sensors and noise, plugins, robot models, cloud simulation and lots more [8].

ns3(Network Simulator 3): which is a discrete-event network simulator for internet systems targeted primarily for research and educational use. This consists of several Wi-Fi models to simulate network conditions.Also includes different models which are sufficiently realistic to allow ns-3 to be used as a realtime network emulator, interconnected with the real world [9].

ROS2(Robotic Operating System 2): is a set of software libraries and tools for building robot applications. ROS2 consists of open-source tools in developing state-of-art-algorithms to powerful developer tools. The tools are build on several concepts such as graphs, nodes, client libraries [10]. This is extremely useful in integrating the tools and provides a stable control in controlling and handling the tools.

B. Existing Integrated Simulation Frameworks

Initial research often prioritized specific aspects of UAV operations, such as path planning or flight dynamics. Swarmlab, which is a MATLAB based testbed designed for the rapid comparison of swarm algorithms which effectively visualizes metrics like swarm connectivity and safety. It relies on simplified point mass dynamics or basic quadcopter models, lacking the hardware level fidelity [11]. On the other end tools such as RotorS provide high fidelity physics simulations within the Gazebo environment [12] capable of simulating sensors like IMUs and odometry and it mainly focusses on aerodynamic control on multi-UAVs rather than focussing on networking in swarm simulation.

To address the limitations of isolated tools, integrated frameworks have been developed. Recent literature presents three significant advancements in this field.

Integration of heterogeneous tools: it is a multi layer architecture integrating Gazebo, ArduPilot and NS-3 using a custom TCP based synchronization tool called $GZUAVCHANNEL$. This approach introduced latency and lacked support for real-time telemetry data paths [13].

FlyNetSim: bridges the gap between ArduPilot and ns-3 without the overhead of virtual machines has been the main goal of introuduction. The core innovation is the use of a middleware based on Zero Message Queue (ZMQ), which is a light weight message library. This intentionally avoids heavy weight physics engines like Gazebo to maintain scalability [14].

DronNS-3: is built on the need for usability, a framework designed to lower the barrier to entry for UAV research. This emphasized and focuses more on making ease of mission planning. It introduces a simplified python-based autopilot library that abstracts MAVLink messages into high level commands [5]. Although this provides an easy setup visualization of the context could be indentified as a drawback.

GazeboNS3: could be identified as the most recent advancement. It proposes a Digital Twin system that combines Gazebo, ArduPilot and ns-3. This system uses a plugin to communicate between ArduPilot SITL and Gazebo which acts as a shim [6].

C. Gaps in the Literature

The analyzed litreature highlights that current simulators primarily focus on routing protocols or point-to-point telemetry. There is a noted absence of frameworks that natively support or provide libraries for swarm simulations adding clustering. Clustering involves UAVs dynamically forming groups and electing a 'cluster head' to aggregate and process data. ArduPilot does not support the logic neither does ns-3. DronNS3 provides a framework for swarming, it leaves the implementation of distributed decision making entirely to the user.

A significant gap exists regarding the middleware used to interconnect simulation components all FlyNetSim, DroNS-3 and GazeboNS3 doesn't specifically uses a middleware. Only DroNS-3 uses MAVROS but currently relies

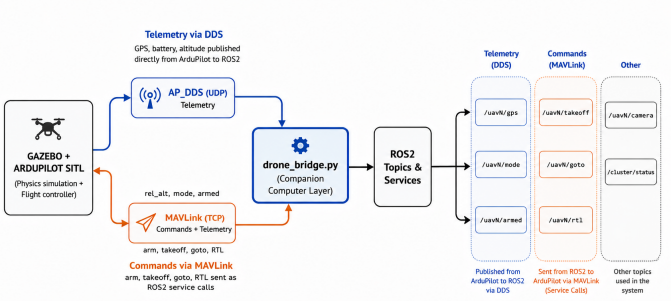


Fig. 1. Integration of ROS2 and ArduPilot using dual channels

on DroneKit and custom scripts [5]. Consequently, lack of literature could be found regarding simulation frameworks that integrates ROS2 as the primary orchestration layer.

This project proposes to bridge this gap by integrating the tools ArduPilot(SITL), Gazebo, ns-3 and ROS2 creating a more modern and standardized architecture than the custom middleware that is capable to carry out multi-UAV swarm simulations.

III. System Architecture and Methodology

A. Environment Setup

ArduPilot SITL is integrated with Gazebo Classic 11 using the official *ardupilot_gazebo* plugin. This plugin acts as a shime between the gazebo physics engine and ArduPilot's flight dynamics model. Network simulation is implemented using ns-3 (3.38) which models the network environment. ROS2 is used as the middleware in integrating these tools. Sending of the location coordinates of each UAV, to ns-3 is done via ROS2. Sending of mission commands to each UAV is fulfilled by ROS2 which highlights the importance of ROS2 as a middleware.

B. Dual-Channel ROS 2 Middleware Integration

ArduPilot and ROS2 communication is performed using ArduPilot's native DDS support (AP_DDS), instead of the traditional MAVROS which caused compatibility issues with ArduPilot and Gazebo Classic 11. The integration uses a dual-channel communication as illustrated in Figure 1. **a)** AP_DDS over UDP: ArduPilot publishes telemetry data such as GPS positions and altitude directly into via ROS2 via the DDS protocol. A *micro_ros_agent* instance runs per UAV, listening on its assigned UDP port and bridging AP_DDS topics into the ROS2 ecosystem. **b)** MAVLink over TCP: Mission commands such as arm, takeoff, go-to-waypoint and RTL are sent from ROS2 to ArduPilot via MAVLink TCP connections on ports assigned to each UAVs. ROS2 packages are organized as an *ament_python* package called the *uav_controller*. All the missions are commanded using modules that contains inside a package. Companion computer bridge nose which initiates one instance per UAV is also added as a module in the same package.

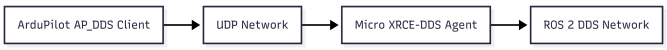


Fig. 2. Communication flow from ArduPilot AP_DDS client to ROS2 DDS network

C. ROS2 - ArduPilot Communication Architecture

MAVLink is one of the communication methods used between ArduPilot SITL and external ground control or monitoring applications. It carries vehicle information such as flight mode, GPS position, altitude, parameter values and system messages. MAVLink also deliver commands to the flight controller, including arming commands, flight mode changes, mission uploads and parameter requests.

Each UAV runs a separate ArduPilot SITL instance and therefore requires its own MAVLink connection to exchange telemetry. Individual TCP ports are assigned to each vehicle so that messages from different ArduPilot instances are not mixed. These ports are provided through the *SERIAL0* interface of each ArduPilot SITL instance. As an example, a monitoring application can connect UAV1 and UAV2 through Port1 and Port2 respectively, where these ports are assigned explicitly to each UAVs. Each ArduPilot process runs inside its corresponding Linux network namespace. The loopback address is therefore, independent within every UAV namespace. A process running inside UAV1 can connect to it's assigned IP address without interfering with the loopback interfaces or MAVLink connections of other UAVs. The MAVLink connection for UAV1 can be represented as,
Ground-control application $\longrightarrow$ TCP Port $\longrightarrow$ ArduPilot SITL UAV1.

DDS is used to exchange data between ArduPilot and ROS2 within the multi-UAV simulation. Through this connection, information generated by each flight controller becomes available on ROS2 topics. It also allows ROS2 nodes to send selected commands and service requests back to the UAVs. Ardupilot includes an internal DDS component called *AP_DDS*, which operates as a Micro XRCE-DDS client. A complete DDS implementation can require considerable memory and processing resources. Micro XRCE-DDS messages produced by ArduPilot and converts them into standard DDS data that ROS2 nodes can use. Messages travelling from ROS2 toward the flight controller are converted in the opposite direction before being returned to ArduPilot. The basic communication flow of ROS2 and ArduPilot is represented in Figure 2.

DDS communication is enabled on each vehicle using the *DDS_ENABLE* parameter. The destination IP address is set to 10.42.0.10, while the UDP destination port is selected according to the UAV. All three vehicles use DDS domain ID0, allowing their agents and the ROS2 nodes to participate in the same DDS domain. Namespace support is also enabled in ArduPilot so that each vehicle receives a separate ROS2 topic prefix.These prefixes avoid topic name conflicts between the vehicles. Without separate namespaces, all ArduPilot instances corresponding to each

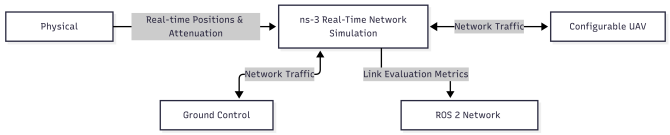


Fig. 3. Network architecture

UAV.

D. Network Simulation Architecture

Reliable communication is an essential requirement in multi-UAV systems since mission coordination, telemetry exchange, control commands, collaborative decision making, and dynamic clustering depend on the availability and quality of wireless links. The network architecture supports a configurable number of UAVs and for the initial implemenatation only three UAVs were used, where this configuration was selected according to the computational capacity of the development computer. Figure 3. represents the high level architecture of ns3 simulation. Without ns-3 integration, applications running on the same computer would communicate directly through the host operating system by-passing the network which would have made the network extremely low in latency and would not experience realistc wireless effects. To overcome this limitation, each simulated node is placed inside an isolated Linux network environment. Virtual Ethernet interfaces, linux bridges, TAP interfaces and the ns-3 TapBridge mechanism connect the Linux applications to simulated ns-3 wireless devices. As a result, actual MAVLink, DDS, ROS2, telemetry and control packets pass through the ns-3 wireless channel instead of by-passing the network.

The network uses a wireless channel configuration between the ground control station and the UAVs inside the ns-3 simulation. Its main purpose is to reproduce the effects caused by a real wireless link. The wireless channel is not configured as fixes or ideal, instead it changes continously according to the movement of the UAVs and the surrounding Gazebo environment. Distance based propagation loss, nakagami fading and a dynamic obstacle loss model is added to simulate the network.

IV. Implementation Details

A. ArduPilot SITL and Gazebo Interoperability

ArduPilot SITL and Gazebo uses the official *ardupilot_gazebo* plugin. Each UAV model in Gazebo loads the *libArduPilotPlugin.so* with its SDF definition, which exhanges motor commands and sensor data (GPS, IMU, barometer) between the two simulators via UDP. Each ArduPilot SITL instance additionally exposes a MAVLink TCP server on its assigned ports for each UAV which commands and telemetry are exchanged through these ports. The iris quadrotor model is used as the base UAV model. To operate in the multi-UAV simulation each UAV is assigned with a unique index, -I flag when launching ArduPilot SITL. Each UAV has its own ArduPilot parameter file which configures the DDS settings, arming checks and other SITL specific parameters.

The iris quadrotor model was used as the base UAV model. Each UAV model was configured to include a small 2D gimbal camera with the *libgazebo_ros_camera.so* plugin, which publishes live camera images directly to ROS2 topics.

B. Real-World Georeferenced Environment Modeling

Environment modelling could be done to make the physical environment of the simulation more realistic and to run the simulation in a custom built based on a real environment. As a representation the premises of the faculty of Engineering, University of Ruhuna, Galle Sri Lanka was modelled. The buildings and terrain were modeled using Blender and exported as COLLADA.dae file. The gazebo world assembles these exported meshes into a complete static environment by referencing each .dae file as both a visual mesh and a collision mesh, allowing the physics engine to interact with the building geometry during simulation.

A key requirement for the faculty environment was that UAV GPS positions reported within the simulation should correspond to real world coordinates on the actual faculty premises. This was achieved by configuring the spherical coordinates block inside the .world file which sets the GPS origin of the simulation to a default value.The world file uses the ODE physics engine, with solver parameters tuned specifically for UAV simulation. The configuration inlcudes several parameters including atmospheric parameters such as gravity and wind conditions.

C. Network Namespaces and TAP Device Bridging

Linux network namespaces provide isolated virtual networking environments for the ground control station ans each simulated UAV. The platform creates four namespaces: *gcsns, UAV1, UAV2 and UAV3*. Each namespace maintains it's own network interfaces, IP addresses, routing table, socket space and networkstack state. Consequently, every UAV behaves as an independent network node, similar to a separate onboard computer in a physical multi-UAV deployment.

Within each namespace, a simulated wireless interface is exposed. Virtual ethernet pairs and linux bridges carry packets between the simulated wireless interface and the corresponding TAP device. The TAP interfaces connect the linux networking environment to the ns-3 wireless devices through the integration layer. When a UAV transmits data to the ground control station, the packet first leaves the UAV namespace through its simulated wireless interface and entering ns-3. The simulator applies the configured network and the processed packet is delivered to the destination TAP device and then forwarded into the recieving namespace. Figure 4 shows the exact path of packet flow starting from a namespace moving towards the ns-3 wireless network.

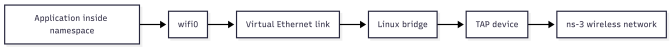


Fig. 4. End-to-end packet path from a namespace application to the ns-3 wireless network

Virtual ethernet interfaces connect each isolated network namespaces to the host networking environment. A virtual ethernet connection is created as a linked interface pair: one end point is moved into the relevant namespace, while the other remains in the root network namespace. A frame transmitted through either endpoint immediately becomes available at the opposite endpoint, enabling communication between the isolated node environment and the host.

D. The Virtual Companion Computer Layer

A key component of the implementation is *drone_bridge.py* which acts as the virtual companion computer for each UAV. One instance of this node runs per UAV and handles all communication between the higher-level mission logic and the ArduPilot SITL instance. This is the single place where MAVLink behavior and message formatting is managed. The node subscribes to AP_DDS telemetry topics published by *micro_ros_agent* and re-publishes simplified, namespace isolated telemetry, so mission nodes never have to deal with raw messages directly.

V. Experimental Results

A. Multi-UAV Flight and Synchronization

This section establishes that the coupling between Gazebo and the ArduPilot (SITL) instances is sound, and that the fleet executes a commanded multi-vehice mission accurartely. A completer `city_mission` run was flown with three vehicles at assigned cruise altitudes of 60, 40 and 50 m comprising is commanded waypoints over approxmimately 210 s, a circular patrol of the city centre by the designated head and two indpendent sweep pattens by the member.

1) Physics autopilot coupling: The position reported by each vehicle is produced by ArduPilot's state estimator, which is driven by the Gazebo flight dynamics model, the world pose is read retractory from the Gazebo model state. Agreement between the two is therefore direct evidence that the loop between the physics engine and the autopilot is genuinely closed rather than merely running. The transform between geoeeditc and world coordinates was obtained by least squares, fit to the recorded data so that the residual measures a real diagreement between the two sources rather that an artifact of an assumed constant.

2) Fleet synchronisation: The mission holds all three vehicles at thread barriers between phases, so the spread in the times at which they receive their next waypoint is the observable synchronisation error. For the three commanded waypoints that follow a common barrier, that spread was 2,4 and 5 ms respectively. Once released the vehicles are commanded within a few milliseconds of one another. Beyond that point the vehicles execute differing numbers of sweep legs, so waypoint index no longer corresponds to a share barrier and the comparison ceases to be meaningful. Arming was less tightly coupled at 5.42 s between the first and last vehicle, because each arming sequence waits independently on it's own pre-arm checks and GPS lock.

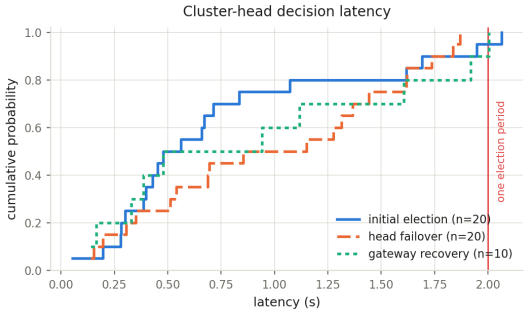


Fig. 5. Cluster head decision latency

Vertical separation, whic is the mission's only collision avoidance mechanism, averaged 8.58 m during the patrol phase against a 10 m design separation. It is not maintained throughout, during the simultaneous climb and again during the coordinated return to launch, the vehicles pass through one another's altitude bands and the minimum separation approaches zero. Altitude stratification is therfore an effective separation strategy at cruise but not during shared vertical transitions, and a deployment would reqire but not during shared vertical transitions, and a deployment would require an explicit deconfliction rule for those spheres.

B. Clustering and Telemetry Aggregation

The clustering layer was evaluated by driving the unmodified `dynamic_cluster_manager` node with a scripted metric feed that substitutes for ns-3. This isolates the election logic against known inputs, and allows every expected value to be recomputed independently from the documented weighted score rather that being read back from the node under test. Unless stated otherwise, each figure reports the mean with a 95 % confidence interval and the sample size.

1) Control-place responsiveness: Figure 5 shows the empirical cumulative distribution of three control plane decisions. The initial election from a gateway less start, failover after the cluster head loses its uplink to the ground control station and recovery after the entire fleet is blinded and then restored. The measured latencies are 0.75 ± 0.28 s, 0.98 ± 0.28 s and 0.91 ± 0.52 s respectively and no trial exceeded the 2 s election period.

The distributions are approximately uniform over $[0, 2]$ s, which is the expected signature of a periodic decision timer sampled at an arbitrary phase. The response time of the clustering layer is governed by the election period rather than my the cost of evaluating the score,

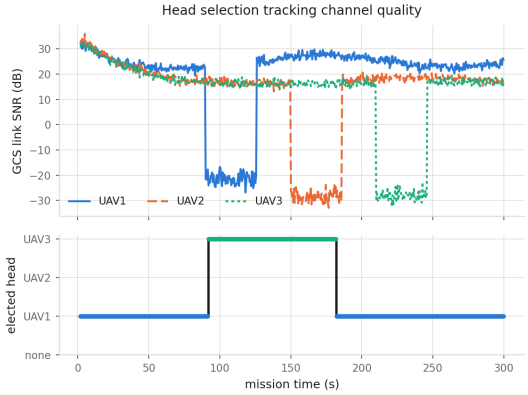


Fig. 6. Head selection tracking channel quality

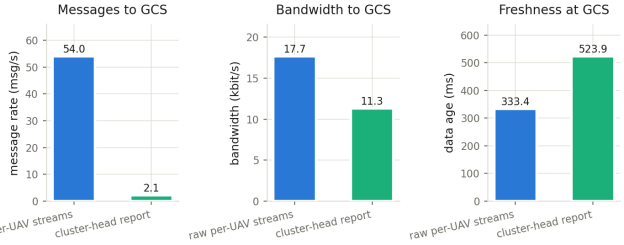


Fig. 7. Aggregation test

so responsiveness scales directly with that period. To obtain this distribution the step change was applied at a uniformly randomised offset within the election period, applying it on a period boundary instead reports the worst case in every trial and understates the achievable responsiveness.

Across 40 randomised link matrices, the score components published by the node agreed with the independent reimplementation to within the resolution of the 32-bit transport ($< 10^{-6}$), and the candidate eligibility flag agreed in 120 of 120 cases. In all 20 initial election trials the elected head was the analytically correct one and in all 20 failover trials the emitted event carried the reason `primary_link_failure`.

2) Cluster stability: Stability was tested under two opposoing conditions with two candidates whose scores differ by less than the configured switch margin with 1.5 dB of Gaussian noise added to every link, the manager performed *zero* handovers over 180 s. When a challenger was then made decisively better, it performed exactly one handover, 5.99 s after the step, against an analytical bound of 6.0 s set by the requirement for three consecutive wins at a 2 s period. The hysteresis therefore suppresses noise driven thrash without preventing a justified transition.

Figure 6 demonstrates the complete decision chain over a 300 s mission. Link SNR is derived from the `city_mission` waypoint geometry through the validated propagation model of Section V-C with three links decays smoothly from approximately 30_,dB to 17_,dB as the fleet transmits away from the ground station, and no handover occurs. The cluster remains stable while link quality degrades uniformly. At $t \approx 125$,s but the head does not revert immediately, because the minimum hold, switch margin and consecutive win conditions must all be satisfied. Reversion occurs at $t \approx 183$,s and $t \approx 210$ s produce no handover at all confirming that the manager reacts to failure of the head's uplink or to a decisively superior challenger and not to every channel event. Over the mission the manager performed two handovers, the mean head tenure was 99.3 ± 39.5 s, and a valid gateway was available for 100 % of the flight (596 of 596 samples).

The manager also degrades safely. When every UAV was blinded from the ground station, the published status became `NO_GATEWAY` and all roles became `DISCONNECTED` in 10 of 10 trials. When the metric feed was stopped entirely, the manager ceased publishing decisions 4.0 s later, within the 5 s staleness timeout rather than continuing to elect on stale data.

3) Telemetry aggregation: Figure 7 compares the cost of delievering that fleet's telemetry to the ground station by two routes on identical input. The fifteen independent per UAV streams that the bridge layer publishes today and a single periodic fleet report at the elected head by the aggregator test node. Bothe were measured over a 60 s window with an emulated fleet publishing at the rates the bridge layer actually produces (position, battery, mode armed at 1 Hz heartbeat).

Aggregation at 2 Hz reduces the message rate reaching the ground station from 54.0 to 2.1 messages s^{-1}, a factor of 25.3 and the offered bandwidth from 17.7 to 11.3 kbit s^{-1}, a reduction of 36.0 %. The price is staleness: the mean age of the value held at the ground station rises from 333 ms to 524 ms, an increase of 191 ms. Transport of the report itself is negligible by comparison at 0.81 ± 0.125 ms. Reducing the report rat to 0.5 Hz increases the message rate reduction to 101× and the bandwidth reduction to 6.3×, at the cost of 744 ms of additional staleness, which characterises the trade off available to a mission desginer.

The asymmetry between the two reductions is important. The byte saving is modest because the report still carries every field; the large gain is in message count. On a shared 802.11n medium each frame incurs a fixed contention, preamble and acknowledgement overhead independent of payload size, so for telemetry messages of a few tens of bytes the frame count rather than the byte count dominates channel occupancy. The aggregator itself cost 1.10 % of a single core and 82.4 MB of resident memory.

C. NS-3 Network Traffic Validation

This section establishes that the wireless channel through which the framework carries its traffic reproduces the physics it claims. ns-3 was driven with a scripted position and obstacle feed standing in for Gazebo, so that geometry is known exactly, and every expected value below is computed from first principles independently of the simulator. Each operating point was repeated over five

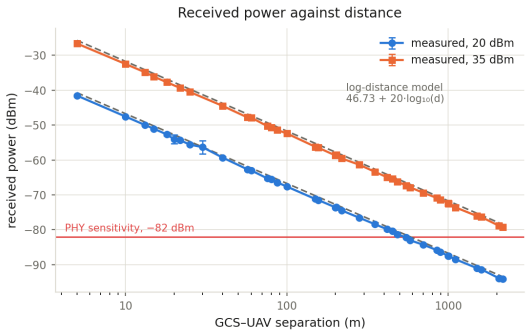


Fig. 8. Received power against GCS separation

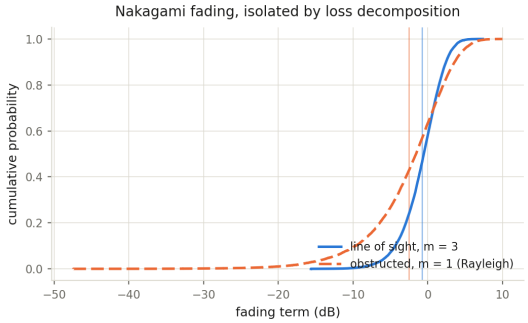


Fig. 9. Nakagami fading isolated by loss decomposition

RNG streams, since a single stream replays one fading realisation and understates variance.

1) Distance based propagation loss: Figure 8 shows mean received power against ground station to UAV separation from 5 to 2179 m at both transmit powers, against the closed form log distance prediction $L(d) = 46.73 + 20\log_{10}(d)$ dB. A least squares fit to the measured means returns a slope of -19.999 dB per decade at both powers, against the -20 dB dBm and 0.0014 dB over 36 bins at 35 dBm. The two curves are parallel and separated by exactly the 15 dB difference in transmit power.

The measured means lie consistently below the deterministic prediction by approximately 0.78 dB. This offset is the mean of the fading term and is not a modelling error which matters when the usable range is quoted. At 20 dBm the deterministic curve crossed the -82 dBm receiver sensitivity at 580.1 m, but because half of all fading realisations fall below that curve the distance at which most frames still decode is 531.3 m. The measured 50 % crossing occurred at 546.7 m, in agreement with the fading corrected prediction rather than the deterministic one. At 35 dBm the corresponding prediction is 2987.6 m, the sweep extended to 2179 m, where 85.4 % of frames still decoded, so the crossing lies beyond the range examined here and no measured value is claimed for that configuration.

2) Nakagami fading: Because the channel chain contains exactly one stochastic stage, the fading term can be isolated exactly by decomposition of the per link loss budget. Figure 9 shows its empirical distribution in both regimes. Under line of sight conditions ($m = 3$) the measured mean and standard deviation are -0.7795 dB and 2.7136 dB, against closed form values of -0.7636 dB and 2.7293 dB. Under obstructed conditions ($m = 1$, Rayleigh) they are -2.4943 dB and 5.6012 dB against -2.5068 dB and 5.5700 dB. All four moments agree to better than 0.04 dB. A Kolmogrov-Smirnov test of the linear fading powr against the gamma distribution it should follow gives $D = 0.0056$, $p = 0.62$ for the line of sight regime and $D = 0.0057$, $p = 0.60$ for the obstructed regime, over 17 970 samples each where neither i rejected at the 5 % level.

The obstructed distribution exhibits a substantially longer lower tail, reaching -4.7 dB, and it is this behaviour that causes shadowed links to fail intermittently rather than cleanly. The fading statistics are identical at both transmit powers, as expected for a multiplicative process evaluated on identica random streams.

VI. Discussion

A. Scalability and Fidelity Trade-offs

The results indicate that the dominant cost of the higher level application layer is not computational. The clustering manager reproduces its documented score to the resolution of the transport, and the telemetry aggregator sustains a three-vehicle fleet for 1.10 % of a single core and 82.4 MB of memory. The clustering layer does cost is latency, and that cost is structural, because decisions are taken on a fixed period, repsonsiveness is bounded below by that period regardless of available computation.

A comparable trade-off governs aggregation. Collapsing fifteen telemetry streams into one head orginated report remoes 96 % of the frames offered to the channel but adds 191 ms to the age of the data at the ground station, and the exchange rate between the two is set directly by the report period. Neither quantity can be improved without worsening the other, so the framwork exposes the report rate a submission

B. Limitations

First, the fading process is drawn independently at each evaluation and is therefore uncorrelated in time. The marginal distribution is correct, as Fig. 9 establishes, but burst errors arising from correlation over the channel coherence time are under-represented.

Second, the mission scenario in Fig. 6 is a replay of the real waypoint geometry through the validated propagation model with scripted occlusion episodes, rather than a closed-loop flight. It demonstrates that the clustering layer responds correctly to a realistic sequence of channel conditions; it does not by itself establish the behaviour of the coupled physical and network simulation.

Third, and most significantly, the results in Section V-C validate the channel model rather than end-to-end transport. They establish what the wireless channel does to a link budget, not what it does to MAVLink and

DDS packet delivery under load. Measurement of end-to-end latency, packet loss and achieved bandwidth for real protocol traffic traversing the TapBridge data path remains to be completed, and the throughput reported by such measurements will additionally be bounded by the real-time forwarding capacity of the host rather than by the simulated channel alone.

VII. Conclusion and Future Work

This work set out to build a simulation framwork in which the flying and the networking of a multi-UAV system are modelled together rather than separately. Four open-source tools were carefully integrated into one platform, ArduPilot SITL flies the vehicles, Gazebo provides the physical world, ROS2 carries commands and telemetry as the middleware layer and ns-3 carries the traffic over a simulated wireless channel. Two additions distinguish the platform from earlier work. The first is a dual channel middleware, in which commands travel to each UAV over MAVLink while telemetry returns over DDS, so that each direction uses the protocol suited to it. The second is a clustering layer, in which the vehicles choose one of their own to act as a cluster head and that head collects the fleet's telemetry.

Finally the software that woulr actually fly on a UAV is light. Excludint the parts that exist only to simulate the world. The architecture is not merely a simulation construct, the coordination and communication layers would fit on the class of small computer that UAVs already carry. Together, these results show that the four tools can be combined into a single platform in which a mission, the rado links it depends on, and the onboard software cost of running it can all tbe observed at once.

A. Future works

Several limitations remain and each points to a clear next step. *Making the cluster head carry traffic.* At present the clustering layer decides which vehicle should be the head and publishes that decision, and the head does collect telemetry, but network traffic is not forced to travel through it. All vehicles share a single wireless network, so any two that can hear each other communicate directly. Making the decision bind on the traffic, either by installing routes on each vehicle or by adding a routing protocol inside NS-3, would turn clustering from a coordination scheme into a genuine relaying scheme, and would allow the benefit of clustering to be measured in delivered packets rather than in message counts.

Larger fleets. All results here use three vehicles. The resource measurements indicate where the limit lies: the physics engine alone consumed 108 % of a core, while the per-vehicle software cost grew at only 13.5 % of a core. Scaling up on one machine will therefore be constrained first by Gazebo. Because the per-vehicle software divides cleanly across machines while the physics engine does not, distributing the simulation across several hosts is the natural way to grow the fleet.

References

[1] F. Nex and F. Remondino, "Uav for 3d mapping applications: a review," *Applied Geomatics*, vol. 6, pp. 1–15, 11 2013.

[2] T. Tomić, K. Schmid, P. Lutz, A. Dömel, M. Kassecker, E. Mair, I. Grixa, F. Ruess, M. Suppa, and D. Burschka, "Toward a fully autonomous uav: Research platform for indoor and outdoor urban search and rescue," *IEEE Robotics & Automation Magazine*, vol. 19, pp. 46–56, 09 2012.

[3] G. Skorobogatov, C. Barrado, and E. Salamí, "Multiple uav systems: A survey," *Unmanned Systems*, vol. 08, pp. 149–169, 04 2020.

[4] H. Shakhatreh, A. Sawalmeh, A. Al-Fuqaha, Z. Dou, E. Almaitta, I. Khalil, N. Othman, A. Khreishah, and M. Guizani, "Unmanned aerial vehicles (uavs): A survey on civil applications and key research challenges," *IEEE Access*, vol. 7, pp. 48 572–48 634, 04 2019.

[5] P. Hall, J. Diller, A. Moon, and Q. Han, "Drons-3: Framework for realistic drone and networking simulators," 06 2023, pp. 39–44.

[6] D. Cauwe, C. Wang, and Q. Han, "Gazebons3: A digital twin system for uav swarms," 09 2025, pp. 699–704.

[7] ArduPilot, "Ardupilot documentation," https://ardupilot.org/, accessed: Aug. 5, 2026.

[8] Open Source Robotics Foundation, "Gazebo simulator," https://classic.gazebosim.org/, 2020, accessed: Aug. 5, 2026.

[9] nsnam, "ns-3 network simulator," https://www.nsnam.org/, accessed: Aug. 5, 2026.

[10] Open Robotics, "Ros 2 documentation," https://docs.ros.org/en/foxy/index.html, 2026, accessed: Aug. 5, 2026.

[11] E. Soria, F. Schiano, and D. Floreano, "Swarmlab: a matlab drone swarm simulator," 05 2020.

[12] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, *RotorS – A Modular Gazebo MAV Simulator Framework*, 01 2016, vol. 625, pp. 595–625.

[13] E. Marconato, M. Rodrigues, R. Pires, D. Pigatto, L. Querino Filho, A. Pinto, and K. Castelo Branco, "Avens – a novel flying ad hoc network simulator with automatic code generation for unmanned aircraft system," 01 2017.

[14] S. Baidya, Z. Shaikh, and M. Levorato, "Flynetsim: An open source synchronized uav network simulator based on ns-3 and ardupilot," 10 2018, pp. 37–45.

G.P.W.P.Wijerathne following the B.Sc. degree in Electrical and Information Engineering from the University of Ruhuna, Galle, Sri Lanka, in 2026.

J.H.A.D.C.Epaladeniya following the B.Sc. degree in Electrical and Information Engineering from the University of Ruhuna, Galle, Sri Lanka, in 2026.

K.A.G.S.Randil following the B.Sc. degree in Electrical and Information Engineering from the University of Ruhuna, Galle, Sri Lanka, in 2026.

W.P.A.N.Fernando following the B.Sc. degree in Electrical and Information Engineering from the University of Ruhuna, Galle, Sri Lanka, in 2026.